CS315 Internet Programming – Fall 2026

HW3 Mini API-Powered Message Board with TypeScript

Total: 10 points

Weight: 10% of the Final Course Grade

Purpose of this assignment:

Build a small API-driven web application that combines the skills from recent lectures: TypeScript types and interfaces, Fetch, JSON, GET and POST requests, form validation, dynamic DOM updates, and static deployment to ICE.

1. **A starter package will be provided on Brightspace:**

CS315_Assignment3_Starter.zip

starter/

index.html

style.css

app.ts

README.txt

2. **Application Theme (No functional implementation related).**
   - Mini Mission Board
   - Campus Bulletin Board
   - Gaming Quest Board
   - Student Club Board
   - Project Idea Board
   - Mini Social Feed

3. **Required API**
   Use JSONPlaceholder for this assignment, like we did in the course slides:

   GET https://jsonplaceholder.typicode.com/users

   GET https://jsonplaceholder.typicode.com/posts?userId=USER_ID

   POST https://jsonplaceholder.typicode.com/posts

   - **Important Note About JSONPlaceholder**
     Important: **JSONPlaceholder *simulates*** the ***creation*** of ***new records***. When you send a POST request, **it returns a response that *looks like* the record was created** (including an ID), which is useful for practicing the HTTP request/response workflow. However, the

new record is not permanently stored on the server. ***If you reload the page or send a later GET request, you should not expect the new post to appear***. Persistent storage will be introduced later when we build our own backend and database.

- **Important JSONPlaceholder behavior**
  JSONPlaceholder ***simulates*** successful POST requests. It returns a response that looks like a newly created record, but the new record is not permanently stored on the server. Refreshing or fetching again will not show your newly submitted post.

Q1 — Load Users with TypeScript (2 points)

- Use GET /users to retrieve multiple user records.
- Define and use a User interface with at least id, name, and email.
- Treat the returned data as User[].
- Dynamically add the users to the provided <select> element.
- Display a useful loading/success/error message on the webpage.

Q2 — Select a User and Display Posts (2 points)

- After a user is selected, request that user's posts using /posts?userId=....
- Define and use a Post interface.
- Treat the returned data as Post[].
- Use forEach() and DOM methods such as createElement(), textContent, and appendChild() to display multiple posts.
- Display at least the post title and body for each returned post.

Q3 — Create and Validate a New Post Form (2 points)

- Use the provided form for title and message/body.
- Require the user to select a user before submission.
- Validate that title and message are not empty after trim().
- Provide visible feedback on the webpage. Reuse CSS classes such as .error and .success or create equivalent classes.
- Use preventDefault() so the page is not normally submitted/reloaded.

Q4 — Send a POST Request and Display the Response (2 points)

- Define and use a NewPost interface containing userId, title, and body.
- Create a typed NewPost object from the form values.
- Send it with fetch() using method: "POST".

- Use Content-Type: application/json and JSON.stringify().
- Check response.ok and handle failures.
- Parse the returned JSON and display the returned ID, selected user, title, and message in the webpage (console-only output is not sufficient).

Q5 — Custom Feature, Testing, and Deployment (2 points)

- Add ONE meaningful JavaScript/TypeScript behavior beyond the starter requirements. Examples: keyword filter, post count, character counter, reset/clear button, show/hide user email, simple sorting, highlight selected user, or another reasonable feature.
- A CSS-only color change does not count as the custom feature.
- Test at least three cases: normal GET/display, invalid form input, and successful POST response.
- Deploy the final runnable site to ICE under public_html/assignment3/.
- Submit the deployed URL and required screenshots/documentation to Brightspace.

Minimum TypeScript Requirements

Your app.ts must meaningfully use TypeScript. At minimum, include:

- interface User { ... }
- interface Post { ... }
- interface NewPost { ... }
- At least one **typed** **array** such as User[] or Post[].
- At least one **typed** **function parameter** and/or return type.

** Do not simply rename a JavaScript file from .js to .ts without using types.

For example, there is a loadPosts() function:

```
function loadPosts(userId: number): void {
   ...
}
```

Compile and Run Locally

- Open the assignment folder in VS Code.
- Compile the TypeScript file in VS Code's terminal:

tsc.cmd [SPACE] app.ts

Or,

tsc [SPACE] app.ts in Window's Command Prompt

If you are using Command Prompt, macOS, or Linux and tsc works directly, you may use:

tsc [SPACE] app.ts

- Confirm that app.js is generated.
- Open index.html using Live Server.
- Use Chrome/Edge DevTools Network and Console tabs when debugging Fetch requests.

Deploy to ICE

Your final site must be available from ICE so the TA/grader can test it by URL.

~/public_html/assignment3/

index.html

style.css

app.js

Your public URL should follow this pattern:

http://ice.truman.edu/~USERNAME/assignment3/

- The deployed browser version needs index.html, style.css, and the compiled app.js.
- Your Brightspace ZIP must also include app.ts so the grader can inspect your TypeScript source.
- Use appropriate permissions so the web server can traverse your home directory and read the deployed files.

Off-campus access

When you are off campus, connect to the Truman VPN before accessing or testing university systems when required. Test your deployed URL yourself before submitting it.

Reference Run Screen

Your page does not need to look identical; it only needs to satisfy the required behavior and remain readable.

Mini Mission Board

API-Powered TypeScript Demo

1. Users

Load Users



Choose a user:

-- Select a user --

Load Posts

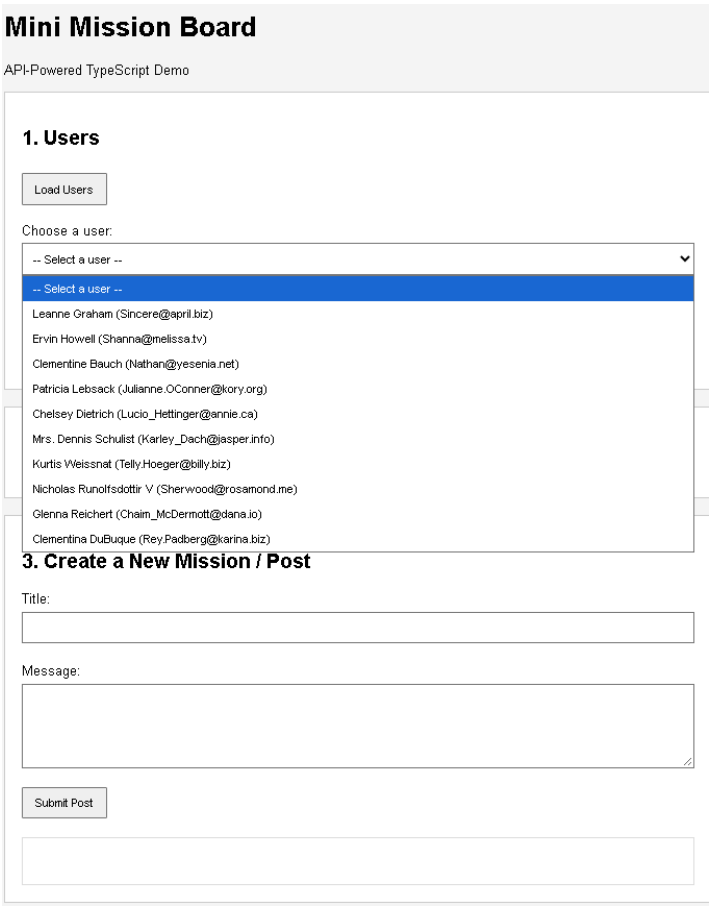
2. Posts

3. Create a New Mission / Post

Title:

Message:

Submit Post



If I choose the 1st user and click the “Load Posts”



Lots of posts will be loaded in:

2. Posts

sunt aut facere repellat provident occaecati excepturi optio reprehenderit

quia et suscipit suscipit recusandae consequuntur expedita et cum reprehenderit molestiae ut ut quas totam nostrum rerum est autem sunt rem eveniet architecto

qui est esse

est rerum tempore vitae sequi sint nihil reprehenderit dolor beatae ea dolores neque fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis qui aperiam non debitis possimus qui neque nisi nulla

ea molestias quasi exercitationem repellat qui ipsa sit aut

et iusto sed quo iure voluptatem occaecati omnis eligendi aut ad voluptatem doloribus vel accusantium quis pariatur molestiae porro eius odio et labore et velit aut

eum et est occaecati

ullam et saepe reiciendis voluptatem adipisci sit amet autem assumenda provident rerum culpa quis hic commodi nesciunt rem tenetur doloremque ipsam iure quis sunt voluptatem rerum illo velit

nesciunt quas odio

repudiandae veniam quaerat sunt sed alias aut fugiat sit autem sed est voluptatem omnis possimus esse voluptatibus quis est aut tenetur dolor neque

dolorem eum magni eos aperiam quia

ut aspernatur corporis harum nihil quis provident sequi mollitia nobis aliquid molestiae perspiciatis et ea nemo ab reprehenderit accusantium quas voluptate dolores velit et doloremque molestiae

magnam facilis autem

dolore placeat quibusdam ea quo vitae magni quis enim qui quis quo nemo aut saepe quidem repellat excepturi ut quia sunt ut sequi eos ea sed quas

If I type this and click the [Submit Post]

3. Create a New Mission / Post

Title:

I am Leanne

Message:

Hi There.

Submit Post

Then, we can see this:



Note: You need to use fetch() to send the message by using POST to remote API and get the returns.

You need to use these returns to ***populate the info to the HTML***

Required Brightspace Submission

- Submit the following:
    - 1 ZIP file: Assignment3_<YourLastName>.zip containing **index.html**, **style.css**, **app.ts**, and **app.js** (plus any additional files you created).
    - 1 PDF or DOCX results file: **Assignment3_Results_<YourLastName>.pdf** or **.docx** containing your deployed ICE URL and screenshots showing:
        - Users loaded and displayed/selected.
        - Posts loaded for a selected user.
        - An invalid form submission with visible validation.
        - A successful POST response displayed in the webpage.
        - Your custom feature working.

AI Assistance and Code Verification

Limited AI assistance may be used only as allowed by the course policy. You are responsible for understanding, testing, and being able to modify everything you submit.

- Do not submit a complete generated application that you cannot explain.
- Do not fabricate screenshots, output, or deployment evidence.
- The instructor may randomly select students for an individual code-verification meeting.
- You may be asked to explain Fetch, response.json(), JSON.stringify(), interfaces, User[]/Post[], forEach(), DOM creation, validation, POST, or to make a small modification to your own code.

Grading Rubric — 10 Points

Area	Points	Full-credit evidence
Q1 — GET users + User interface + dynamic select	2	Users load correctly; typed User/User[] is used; dropdown is created dynamically; messages/errors are handled.
Q2 — Selected user posts + Post interface + DOM	2	Correct userId request; Post/Post[] used; multiple records are rendered with DOM methods.
Q3 — Form validation	2	User/title/body validation works; preventDefault() used; visible feedback is provided.
Q4 — POST + NewPost + response display	2	Correct POST, JSON header/body, JSON.stringify(), response.ok handling, and visible returned result.
Q5 — Custom feature + testing + ICE deployment	2	Meaningful custom behavior; evidence screenshots; working ICE URL; source and compiled files submitted.

Final Checklist

☐ I read and understood the provided starter code before extending it.

☐ My app.ts compiles successfully to app.js.

☐ I use User, Post, and NewPost interfaces meaningfully.

☐ I can load users and posts from JSONPlaceholder.

☐ My form validates invalid input.

☐ My POST request uses JSON.stringify() and application/json.

☐ I display the POST response in the webpage.

☐ I added one meaningful custom behavior.

☐ My ICE deployment works at /assignment3/.

☐ I tested my deployed URL (and used VPN if off campus/required).

☐ My Brightspace ZIP includes both app.ts and app.js.

☐ My results file contains the URL and required screenshots.